Code Explanation Line by Line (Detailed)

import numpy as np

Explanation:

- **Imports NumPy library.**
- **NumPy is used for:**
    - **Arrays**
    - **Matrix operations**
    - **Mathematical calculations**

Without NumPy, handling vectors and matrices becomes difficult.

---

import matplotlib.pyplot as plt

Explanation:

- **Imports Matplotlib library.**
- **Used for drawing graphs and plots.**
- **plt is shortcut name for easier coding.**

X = np.array([[2,3],[3,4],[4,5],[1,1],[2,1],[3,1]])

Explanation:

- **Creates training input dataset.**
- **Each row represents one data point.**
- **Each point contains two features:**
    - **x1**
    - **x2**

Training points:

[2,3]
[3,4]
[4,5]

belong to one class.

And:

[1,1]
[2,1]
[3,1]

belong to another class.

---

y = np.array([1,1,1,-1,-1,-1])

Explanation:

- **Stores output labels.**
- **1 represents positive class.**
- **-1 represents negative class.**

Mapping:

Input	Output
[2,3]	1
[1,1]	-1

---

w = np.zeros(2)

Explanation:

- **Initializes weight vector with zero.**
- **Since input has 2 features, 2 weights are needed.**

Initial weights:

[0. 0.]

---

b = 0

Explanation:

- **Initializes bias value.**
- **Bias helps shift decision boundary.**

---

learning_rate = 0.1

Explanation:

- **Sets speed of learning.**
- **Small value gives stable learning.**
- **Large value may cause unstable learning.**

Training Process

for epoch in range(20):

Explanation:

- **Runs training for 20 iterations.**

- One complete cycle through dataset is called an epoch.

More epochs → better learning.

---

for i in range(len(X)):

Explanation:

- Accesses every training sample one by one.

---

net = np.dot(w, X[i]) + b

Explanation:

- Calculates weighted sum.

Formula:

$$net = w_1x_1 + w_2x_2 + b$$

Example:
If:

w=[1,2]
x=[2,3]

Then:

net=(1×2)+(2×3)

---

y_pred = 1 if net >= 0 else -1

Explanation:

- Activation function.
- Makes prediction based on net value.

Condition:

Net Value	Output
≥ 0	1
< 0	-1

---

w = w + learning_rate * (y[i] - y_pred) * X[i]

Explanation:

- Updates weights if prediction is wrong.
- Main learning step of perceptron.

If:

- Actual output ≠ Predicted output
- Weights are adjusted.

This helps model learn correctly.

---

b = b + learning_rate * (y[i] - y_pred)

Explanation:

- Updates bias value.

Bias adjustment improves decision boundary.

Decision Boundary

x1 = np.linspace(0,5,100)

Explanation:

- Creates 100 evenly spaced values between 0 and 5.

Used for plotting line.

---

x2 = -(w[0] * x1 + b) / w[1]

Explanation:

- Calculates equation of decision boundary line.

Equation:

$$x_2 = -\frac{w_1 x_1 + b}{w_2}$$

This line separates two classes.

---

plt.figure()

Explanation:

- Creates graph window.

---

plt.scatter(X[y == 1][:,0], X[y == 1][:,1])

Explanation:

- Plots positive class points on graph.

---

plt.scatter(X[y == -1][:,0], X[y == -1][:,1])

Explanation:

- **Plots negative class points.**

---

plt.plot(x1, x2)

Explanation:

- **Draws decision boundary line.**

---

plt.xlabel('x1')
plt.ylabel('x2')

Explanation:

- **Labels x-axis and y-axis.**

---

plt.title('Perceptron Decision Boundary')

Explanation:

- **Adds graph title.**

---

plt.show()

Explanation:

- **Displays final graph.**

Viva Questions with Detailed Answers

1. What is Perceptron?

Perceptron is the simplest Artificial Neural Network used for binary classification problems.

---

2. Who developed Perceptron?

Perceptron was developed by Frank Rosenblatt in 1958.

---

3. What is ANN?

ANN stands for Artificial Neural Network.
It is a computer system inspired by the human brain.

---

4. What is binary classification?

Classification into two categories.

Examples:

- **Even/Odd**

- Yes/No
- Spam/Not Spam

---

5. What is decision boundary?

A line or boundary that separates two classes.

---

6. What is weight in perceptron?

Weight determines importance of input features.

---

7. What is bias?

Bias shifts the decision boundary to improve classification.

---

8. What is activation function?

Function that converts net input into output.

---

9. What is learning rate?

Learning rate controls speed of weight updates.

---

10. What is epoch?

One complete training cycle through dataset.

---

11. What is supervised learning?

Learning using labeled training data.

---

12. What is linearly separable data?

Data that can be separated using a straight line.

---

13. Why perceptron cannot solve XOR problem?

Because XOR is non-linear and perceptron creates only linear boundaries.

---

14. Why NumPy is used?

For array handling and mathematical operations.

---

15. Why Matplotlib is used?

For plotting graphs and visualization.

---

16. What does np.dot() do?

Calculates dot product of vectors.

---

17. What happens during training?

Weights and bias are updated to reduce error.

---

18. What is the output of perceptron?

Binary output:

- **+1**
- **-1**

---

19. What is the purpose of decision region graph?

To visually show separation between classes.

---

20. Why is perceptron important?

It is the foundation of deep learning and neural networks.

Conclusion

Thus, we successfully implemented the perceptron learning law using Python and visualized the decision regions graphically. The perceptron correctly classified linearly separable data using weight updates and activation functions.